260429CAM

April 30, 2026

[1]:
```
!mkdir -p ~/work/class_activation_map/data
!ln -s ~/data/* ~/work/class_activation_map/data/
```

[2]:
```
!pip install opencv-python
```

```
Collecting opencv-python
  Downloading
opencv_python-4.13.0.92-cp37-abi3-manylinux_2_28_x86_64.whl.metadata (19 kB)
Requirement already satisfied: numpy>=2 in /opt/conda/lib/python3.12/site-
packages (from opencv-python) (2.2.6)
Downloading opencv_python-4.13.0.92-cp37-abi3-manylinux_2_28_x86_64.whl (72.9
MB)
                         72.9/72.9 MB
15.2 MB/s eta 0:00:0000:0100:01
Installing collected packages: opencv-python
Successfully installed opencv-python-4.13.0.92
```

1.

[3]:
```
import os
import shutil
import scipy.io as sio
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision.transforms as transforms
import torchvision.datasets as datasets
import torchvision.models as models
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt
import numpy as np
import cv2
import torch.nn.functional as F

import xml.etree.ElementTree as ET
from PIL import Image, ImageFile

ImageFile.LOAD_TRUNCATED_IMAGES = True
```

[4]:
```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(device)
```

```
cuda
```

[5]:
```
!wget http://vision.stanford.edu/aditya86/ImageNetDogs/images.tar
!wget http://vision.stanford.edu/aditya86/ImageNetDogs/annotation.tar
!wget http://vision.stanford.edu/aditya86/ImageNetDogs/lists.tar
!tar -xf images.tar -C ~/work/class_activation_map/
!tar -xf annotation.tar -C ~/work/class_activation_map/
!tar -xf lists.tar -C ~/work/class_activation_map/

print("       !")
```

```
--2026-04-28 07:51:56--
http://vision.stanford.edu/aditya86/ImageNetDogs/images.tar
Resolving vision.stanford.edu (vision.stanford.edu)… 171.64.68.10
Connecting to vision.stanford.edu (vision.stanford.edu)|171.64.68.10|:80…
connected.
HTTP request sent, awaiting response… 200 OK
Length: 793579520 (757M) [application/x-tar]
Saving to: ‘images.tar’

images.tar          100%[===================>] 756.82M  14.1MB/s    in 49s

2026-04-28 07:52:46 (15.3 MB/s) - ‘images.tar’ saved [793579520/793579520]

--2026-04-28 07:52:46--
http://vision.stanford.edu/aditya86/ImageNetDogs/annotation.tar
Resolving vision.stanford.edu (vision.stanford.edu)… 171.64.68.10
Connecting to vision.stanford.edu (vision.stanford.edu)|171.64.68.10|:80…
connected.
HTTP request sent, awaiting response… 200 OK
Length: 21852160 (21M) [application/x-tar]
Saving to: ‘annotation.tar’

annotation.tar      100%[===================>]  20.84M  4.18MB/s    in 8.3s

2026-04-28 07:52:55 (2.51 MB/s) - ‘annotation.tar’ saved [21852160/21852160]

--2026-04-28 07:52:55--
http://vision.stanford.edu/aditya86/ImageNetDogs/lists.tar
Resolving vision.stanford.edu (vision.stanford.edu)… 171.64.68.10
Connecting to vision.stanford.edu (vision.stanford.edu)|171.64.68.10|:80…
connected.
HTTP request sent, awaiting response… 200 OK
Length: 481280 (470K) [application/x-tar]
Saving to: ‘lists.tar’
```

```
lists.tar           100%[===================>] 470.00K   310KB/s    in 1.5s

2026-04-28 07:52:57 (310 KB/s) - ‘lists.tar’ saved [481280/481280]
```

!

[6]:
```python
#        (       )
images_dir = os.path.join(os.getenv("HOME"), 'work/class_activation_map/
 ↪Images')   #
mat_dir = os.path.join(os.getenv("HOME"), 'work/class_activation_map')   #     ␣
 ↪

# .mat     (train_list.mat  test_list.mat          )
train_mat_path = os.path.join(mat_dir, 'train_list.mat')
test_mat_path = os.path.join(mat_dir, 'test_list.mat')

# .mat
train_mat = sio.loadmat(train_mat_path)
test_mat = sio.loadmat(test_mat_path)

# train_mat  test_mat    'file_list'  'labels'
train_file_list = train_mat['file_list']
train_labels = train_mat['labels'].squeeze()  # (N,)
test_file_list = test_mat['file_list']
test_labels = test_mat['labels'].squeeze()

#     ImageFolder             ( : stanford_dogs/train, stanford_dogs/test)
base_dir = 'stanford_dogs'
train_dir = os.path.join(base_dir, 'train')
test_dir = os.path.join(base_dir, 'test')

os.makedirs(train_dir, exist_ok=True)
os.makedirs(test_dir, exist_ok=True)

def process_mat_file(file_list_array, split_dir):
    """
    .mat       file_list_array    ,
    split_dir( : train    test)                 .
    """
    num_files = file_list_array.shape[0]
    for idx in range(num_files):
        # file_list_array[idx]    array([<  >])    .
        #   , array([<  >]).item()                .
        file_path = file_list_array[idx][0].item()

        #   bytes
```

```
        if isinstance(file_path, bytes):
            file_path = file_path.decode('utf-8')

        #       : 'n02116738-African_hunting_dog/n02116738_2988.jpg'
        #                 ( : 'n02116738-African_hunting_dog')
        class_folder = file_path.split('/')[0]

        #
        dest_folder = os.path.join(split_dir, class_folder)
        os.makedirs(dest_folder, exist_ok=True)

        #       : Images      file_path
        src_path = os.path.join(images_dir, file_path)
        #       : dest_folder
        dest_path = os.path.join(dest_folder, os.path.basename(file_path))

        #
        if os.path.exists(src_path):
            shutil.copy(src_path, dest_path)
        else:
            print(f"File not found: {src_path}")

print("Processing train set...")
process_mat_file(train_file_list, train_dir)
print("Processing test set...")
process_mat_file(test_file_list, test_dir)

print("Dataset reorganization complete.")
```

```
Processing train set…
Processing test set…
Dataset reorganization complete.
```

[7]:
```
class StanfordDogsDatasetWithBBox(datasets.ImageFolder):
    def __init__(self, root, annotation_root, transform=None):
        super().__init__(root, transform=transform)
        self.annotation_root = annotation_root  #  : '/content/Annotation'
        self.new_size = (224, 224)

    def __getitem__(self, index):
        image, label = super().__getitem__(index)
        path, _ = self.samples[index]

        rel_path = os.path.relpath(path, self.root)
        annot_filename = os.path.splitext(os.path.basename(rel_path))[0]
        annot_folder = os.path.dirname(rel_path)
```

```
        annot_path = os.path.join(self.annotation_root, annot_folder,␣
↪annot_filename)

        # bbox    ( : [ymin, xmin, ymax, xmax])
        bbox = [0.0, 0.0, 0.0, 0.0]

        if os.path.exists(annot_path):
            try:
                #       XML
                tree = ET.parse(annot_path)
                root_xml = tree.getroot()
                #     object     bndbox
                obj = root_xml.find('object')
                if obj is not None:
                    bndbox = obj.find('bndbox')
                    if bndbox is not None:
                        xmin = float(bndbox.find('xmin').text)
                        ymin = float(bndbox.find('ymin').text)
                        xmax = float(bndbox.find('xmax').text)
                        ymax = float(bndbox.find('ymax').text)
                        # XML   <size>
                        size = root_xml.find('size')
                        w = float(size.find('width').text)
                        h = float(size.find('height').text)
                        new_h, new_w = self.new_size
                        # bbox     : [ymin, xmin, ymax, xmax]
                        bbox = [xmin * (new_w / w), ymin * (new_h / h),
                                xmax * (new_w / w), ymax * (new_h / h)]
                    else:
                        print(f"bndbox          : {annot_path}")
                else:
                    print(f"object          : {annot_path}")
            except Exception as e:
                print(f"Error parsing {annot_path}: {e}")
        else:
            print(f"Annotation file not found: {annot_path}")

        return image, label, bbox
```

[8]:
```
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])

#       (   ImageFolder   )
train_dir = os.path.join('stanford_dogs', 'train')
```

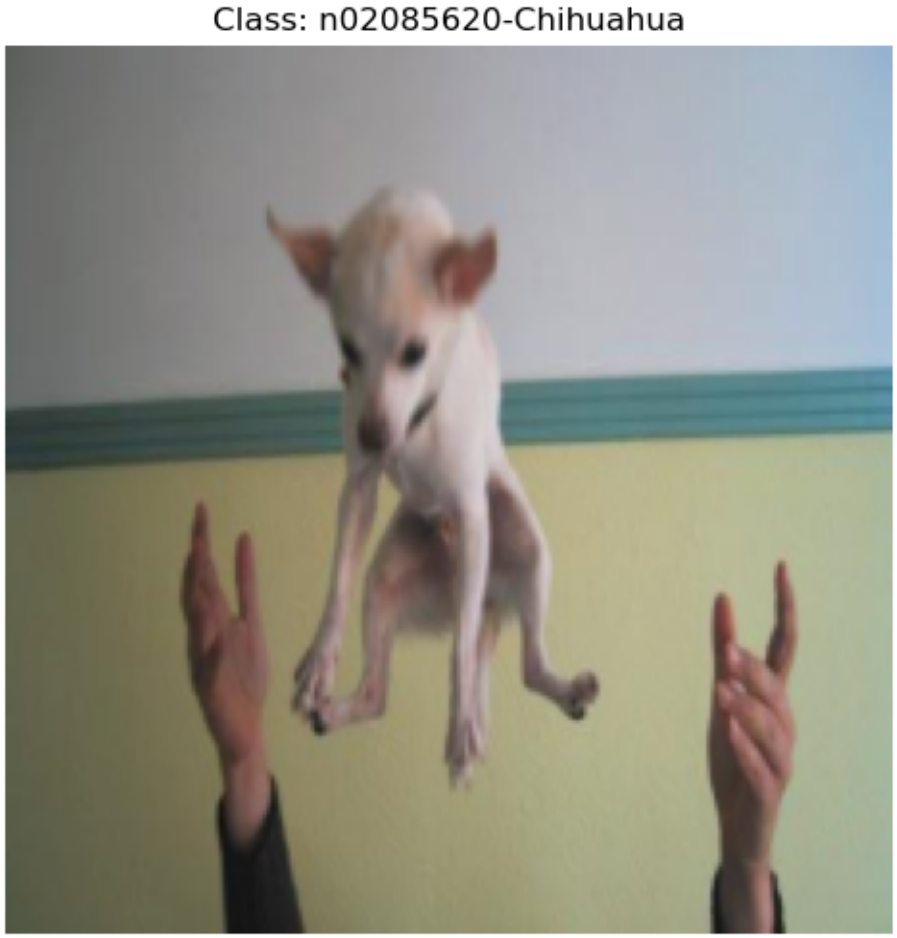
```
test_dir = os.path.join('stanford_dogs', 'test')
# Annotation     ( : '/root/Annotation')
annotation_dir = os.path.join(os.getenv("HOME"), 'work/class_activation_map/
 ↪Annotation')

#        : image, label, bbox
train_dataset = StanfordDogsDatasetWithBBox(root=train_dir,␣
 ↪annotation_root=annotation_dir, transform=transform)
valid_dataset = StanfordDogsDatasetWithBBox(root=test_dir,␣
 ↪annotation_root=annotation_dir, transform=transform)

batch_size = 12
train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True,␣
 ↪num_workers=2)
valid_loader = DataLoader(valid_dataset, batch_size=batch_size, shuffle=False,␣
 ↪num_workers=2)

num_classes = len(train_dataset.classes)
print(f"Number of classes: {num_classes}")
print(f"Train samples: {len(train_dataset)}")
print(f"Test samples: {len(valid_dataset)}")
```

```
Number of classes: 120
Train samples: 12000
Test samples: 8580
```

[9]:
```
def visualize(sample):
    img_tensor, label, bbox = sample

    #
    print("Label index:", label)
    print("Class name:", train_dataset.classes[label])

    #
    mean = np.array([0.485, 0.456, 0.406])
    std = np.array([0.229, 0.224, 0.225])

    # tensor  shape: (C, H, W) -> (H, W, C)
    img_np = img_tensor.permute(1, 2, 0).numpy()
    img_np = std * img_np + mean
    img_np = np.clip(img_np, 0, 1)

    plt.figure(figsize=(6,6))
    plt.imshow(img_np)
    plt.title(f"Class: {train_dataset.classes[label]}")
    plt.axis('off')
    plt.show()
```

[10]:
```
#          (           )
train_sample = train_dataset[0]

visualize(train_sample)
```

```
Label index: 0
Class name: n02085620-Chihuahua
```



[11]:
```
#          (           )
test_sample = valid_dataset[0]

visualize(test_sample)
```

```
Label index: 0
Class name: n02085620-Chihuahua
```

Class: n02085620-Chihuahua

2.

[12]:
```
# Annotation
annotation_path = os.path.join(os.getenv("HOME"), 'work/class_activation_map/
↪Annotation/n02085620-Chihuahua/n02085620_10074')


# XML
tree = ET.parse(annotation_path)
root = tree.getroot()

for elem in root.iter():
    print(f"{elem.tag}: {elem.text}")
```

```
annotation:

folder: 02085620
filename: n02085620_10074
source:
```

```
database: ImageNet database
size:

width: 333
height: 500
depth: 3
segment: 0
object:

name: Chihuahua
pose: Unspecified
truncated: 0
difficult: 0
bndbox:

xmin: 25
ymin: 10
xmax: 276
ymax: 498
```

3. CAM

[13]:
```
model = models.resnet50(pretrained=True)
num_ftrs = model.fc.in_features
model.fc = nn.Linear(num_ftrs, len(train_dataset.classes))
model = model.to(device)
```

```
/opt/conda/lib/python3.12/site-packages/torchvision/models/_utils.py:208:
UserWarning: The parameter 'pretrained' is deprecated since 0.13 and may be
removed in the future, please use 'weights' instead.
  warnings.warn(
/opt/conda/lib/python3.12/site-packages/torchvision/models/_utils.py:223:
UserWarning: Arguments other than a weight enum or `None` for 'weights' are
deprecated since 0.13 and may be removed in the future. The current behavior is
equivalent to passing `weights=ResNet50_Weights.IMAGENET1K_V1`. You can also use
`weights=ResNet50_Weights.DEFAULT` to get the most up-to-date weights.
  warnings.warn(msg)

Downloading: "https://download.pytorch.org/models/resnet50-0676ba61.pth" to
/home/jovyan/.cache/torch/hub/checkpoints/resnet50-0676ba61.pth

100%|      | 97.8M/97.8M [00:00<00:00, 157MB/s]
```

[14]:
```
model
```

[14]:
```
ResNet(
  (conv1): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3),
bias=False)
  (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
```

```
track_running_stats=True)
  (relu): ReLU(inplace=True)
  (maxpool): MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1,
ceil_mode=False)
  (layer1): Sequential(
    (0): Bottleneck(
      (conv1): Conv2d(64, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
      (downsample): Sequential(
        (0): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      )
    )
    (1): Bottleneck(
      (conv1): Conv2d(256, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
    (2): Bottleneck(
      (conv1): Conv2d(256, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
```

```
      (relu): ReLU(inplace=True)
    )
  )
  (layer2): Sequential(
    (0): Bottleneck(
      (conv1): Conv2d(256, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(2, 2), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
      (downsample): Sequential(
        (0): Conv2d(256, 512, kernel_size=(1, 1), stride=(2, 2), bias=False)
        (1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      )
    )
    (1): Bottleneck(
      (conv1): Conv2d(512, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
    (2): Bottleneck(
      (conv1): Conv2d(512, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
```

```
    )
    (3): Bottleneck(
      (conv1): Conv2d(512, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
  )
  (layer3): Sequential(
    (0): Bottleneck(
      (conv1): Conv2d(512, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(2, 2), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
      (downsample): Sequential(
        (0): Conv2d(512, 1024, kernel_size=(1, 1), stride=(2, 2), bias=False)
        (1): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      )
    )
    (1): Bottleneck(
      (conv1): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
```

```
    (2): Bottleneck(
      (conv1): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
    (3): Bottleneck(
      (conv1): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
    (4): Bottleneck(
      (conv1): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
    (5): Bottleneck(
      (conv1): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
```

```
      (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
  )
  (layer4): Sequential(
    (0): Bottleneck(
      (conv1): Conv2d(1024, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(2, 2), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(512, 2048, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
      (downsample): Sequential(
        (0): Conv2d(1024, 2048, kernel_size=(1, 1), stride=(2, 2), bias=False)
        (1): BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      )
    )
    (1): Bottleneck(
      (conv1): Conv2d(2048, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(512, 2048, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
    (2): Bottleneck(
      (conv1): Conv2d(2048, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (conv3): Conv2d(512, 2048, kernel_size=(1, 1), stride=(1, 1), bias=False)
```

```
      (bn3): BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
  )
  (avgpool): AdaptiveAvgPool2d(output_size=(1, 1))
  (fc): Linear(in_features=2048, out_features=120, bias=True)
)
```

4. CAM

[15]:
```
criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.9)
```

[16]:
```
def train_model(model, train_loader, valid_loader, criterion, optimizer,␣
↪num_epochs=2):
    for epoch in range(num_epochs):
        model.train()
        running_loss = 0.0
        correct = 0
        total = 0
        print(f"\nEpoch {epoch+1}/{num_epochs}  ")
        for batch_idx, (images, labels, _) in enumerate(train_loader):
            images, labels = images.to(device), labels.to(device)
            optimizer.zero_grad()
            outputs = model(images)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()

            running_loss += loss.item()
            _, preds = torch.max(outputs, 1)
            total += labels.size(0)
            correct += (preds == labels).sum().item()
            if (batch_idx + 1) % 50 == 0:
                print(f"  Step {batch_idx+1} - Loss: {loss.item():.4f}")

        train_loss = running_loss / (batch_idx + 1)
        train_acc = correct / total
        print(f"Train - Loss: {train_loss:.4f}, Accuracy: {train_acc*100:.2f}%")

        #
        model.eval()
        running_loss = 0.0
        correct = 0
        total = 0
        with torch.no_grad():
            for images, labels, _ in valid_loader:
```

```
                images, labels = images.to(device), labels.to(device)
                outputs = model(images)
                loss = criterion(outputs, labels)
                running_loss += loss.item()
                _, preds = torch.max(outputs, 1)
                total += labels.size(0)
                correct += (preds == labels).sum().item()
        test_loss = running_loss / len(valid_loader)
        test_acc = correct / total
        print(f"Validation - Loss: {test_loss:.4f}, Accuracy: {test_acc*100:.
↪2f}%")
```

[17]:
```
%%time

train_model(model, train_loader, valid_loader, criterion, optimizer,␣
↪num_epochs=2)
```

```
Epoch 1/2
  Step 50 - Loss: 5.1999
  Step 100 - Loss: 2.7993
  Step 150 - Loss: 3.2482
  Step 200 - Loss: 4.0539
  Step 250 - Loss: 3.1958
  Step 300 - Loss: 3.1836
  Step 350 - Loss: 3.5057
  Step 400 - Loss: 3.8292
  Step 450 - Loss: 3.4253
  Step 500 - Loss: 2.4250
  Step 550 - Loss: 2.0622
  Step 600 - Loss: 3.0293
  Step 650 - Loss: 2.1165
  Step 700 - Loss: 1.9058
  Step 750 - Loss: 2.0633
  Step 800 - Loss: 2.3124
  Step 850 - Loss: 2.6055
  Step 900 - Loss: 3.1582
  Step 950 - Loss: 2.2373
  Step 1000 - Loss: 1.6705
Train - Loss: 3.2090, Accuracy: 22.39%
Validation - Loss: 2.6718, Accuracy: 30.99%

Epoch 2/2
  Step 50 - Loss: 1.8281
  Step 100 - Loss: 1.7168
  Step 150 - Loss: 2.6670
  Step 200 - Loss: 2.5560
  Step 250 - Loss: 2.1678
```

```
  Step 300 - Loss: 3.0015
  Step 350 - Loss: 1.4745
  Step 400 - Loss: 2.7707
  Step 450 - Loss: 2.1024
  Step 500 - Loss: 2.6978
  Step 550 - Loss: 1.5799
  Step 600 - Loss: 1.7083
  Step 650 - Loss: 1.1192
  Step 700 - Loss: 1.6116
  Step 750 - Loss: 1.7232
  Step 800 - Loss: 1.8307
  Step 850 - Loss: 3.4686
  Step 900 - Loss: 2.0535
  Step 950 - Loss: 1.8577
  Step 1000 - Loss: 2.4232
Train - Loss: 2.2318, Accuracy: 39.24%
Validation - Loss: 2.6718, Accuracy: 34.38%
CPU times: user 5min 11s, sys: 10.7 s, total: 5min 21s
Wall time: 5min 27s
```

[18]:
```
# Q. '/work/class_activation_map'   'cam_model1.h5'    .

cam_model_path = os.path.join(os.getenv("HOME"), "work/class_activation_map/
 ↪cam_model.pt")

torch.save(model, cam_model_path)
```

5. CAM

[19]:
```
def unnormalize(img_tensor):
    """
    img_tensor: [C, H, W]   (    ,  : normalization   )
    ImageNet              (RGB  )
    """
    mean = np.array([0.485, 0.456, 0.406])
    std  = np.array([0.229, 0.224, 0.225])
    img = img_tensor.cpu().numpy().transpose(1, 2, 0)
    img = std * img + mean
    img = np.clip(img, 0, 1)
    img = np.uint8(255 * img)
    return img
```

[20]:
```
#

image, label, bbox = train_dataset[0]
sample_image = image.unsqueeze(0).to(device)
```

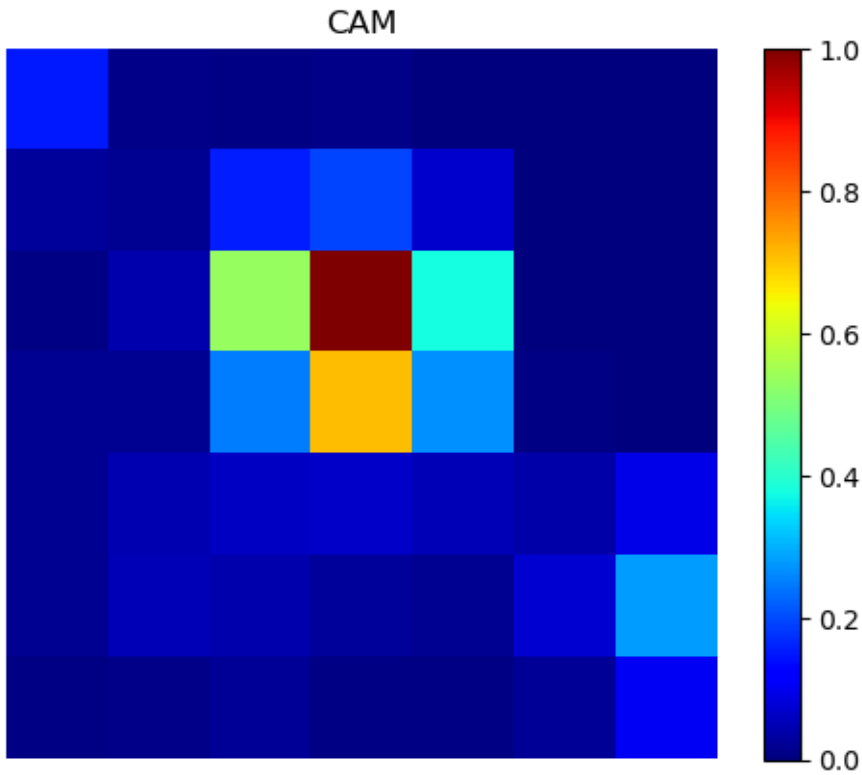
[21]:
```
#

# [[YOUR CODE]]
model = torch.load(cam_model_path, weights_only=False)
```

[22]:
```
def generate_cam(model, image):
    model.eval()
    features = []

    def hook(module, input, output):
        features.append(output.detach())
    hook_handle = model.layer4.register_forward_hook(hook)

    output = model(image)
    hook_handle.remove()

    #         (          )
    pred_class = output.argmax(dim=1).item()

    #    fc layer
    fc_weights = model.fc.weight.data.to(image.device)

    # features[0]: shape [1, num_features, H, W]
    fmap = features[0][0]  #        feature map
    cam = torch.zeros(fmap.shape[1:], dtype=torch.float32, device=image.device)

    # fc    feature map
    for i, w in enumerate(fc_weights[pred_class]):
        cam += w * fmap[i, :, :]
    cam = cam.cpu().numpy()

    # CAM
    cam = np.maximum(cam, 0)
    cam = (cam - np.min(cam)) / (np.max(cam) - np.min(cam) + 1e-8)

    return cam
```

[23]:
```
cam = generate_cam(model, sample_image)
```

[24]:
```
def visualize_cam(cam):
    plt.imshow(cam, cmap='jet')
    plt.colorbar()
    plt.title("CAM")
    plt.axis('off')
    plt.show()
```

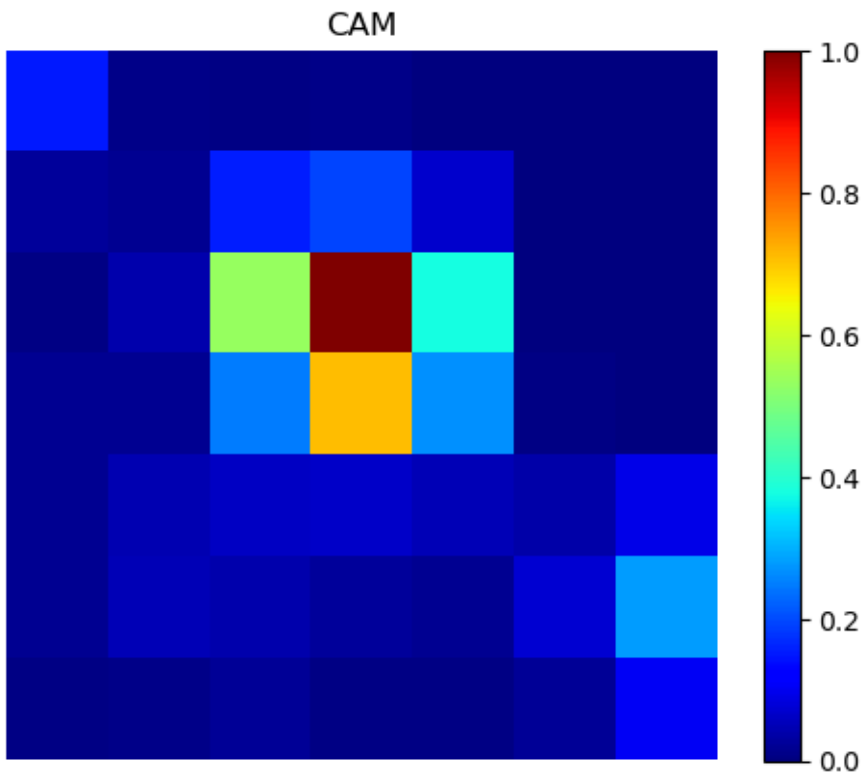
[25]:
```
visualize_cam(cam)
```

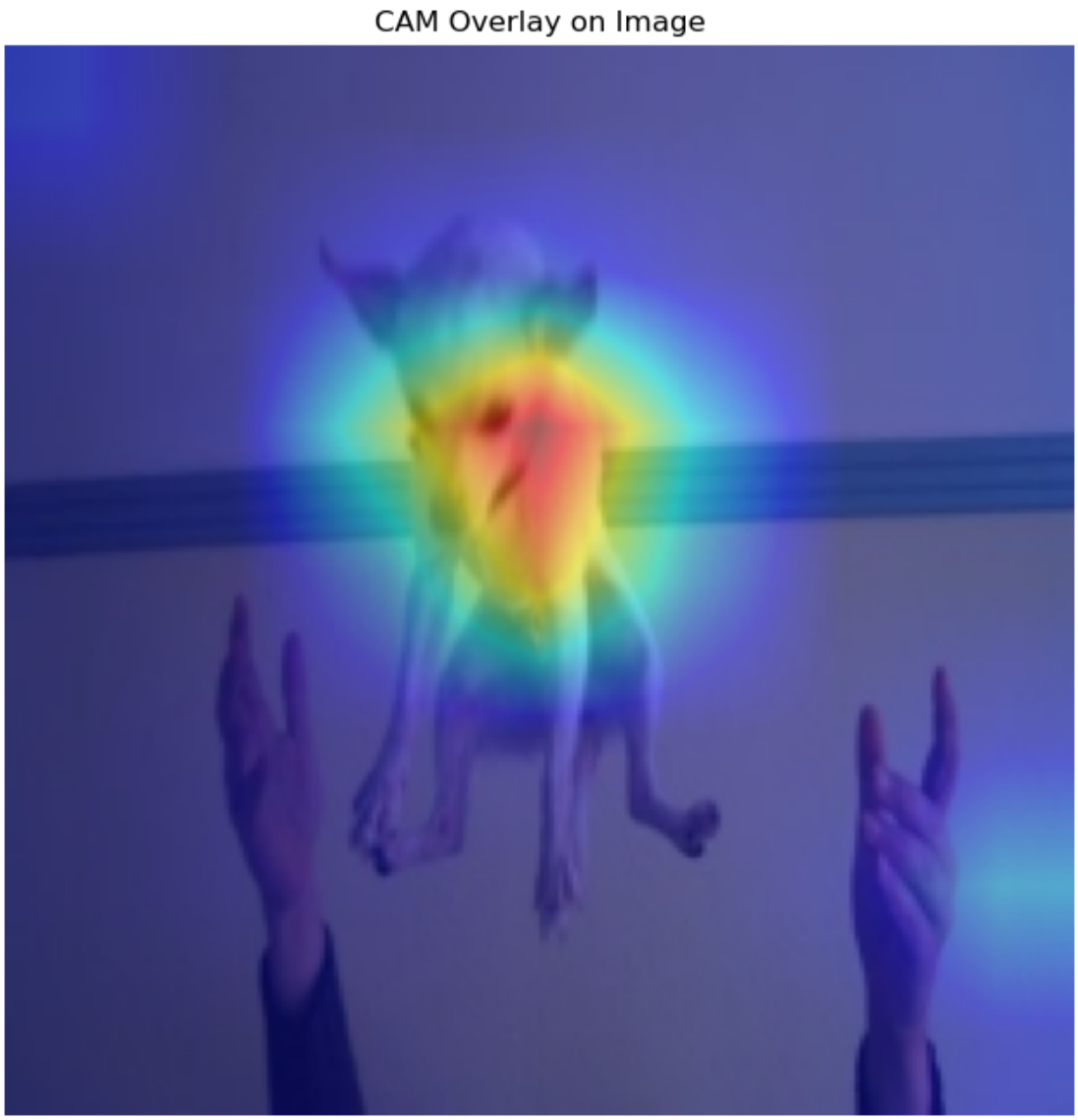


[26]:
```
def visualize_cam_on_image(cam, img, alpha=0.5):
    # CAM
    cam_resized = cv2.resize(cam, (img.shape[1], img.shape[0]))

    # CAM    heatmap    (cv2.applyColorMap  BGR     RGB   )
    heatmap = cv2.applyColorMap(np.uint8(255 * cam_resized), cv2.COLORMAP_JET)
    heatmap = cv2.cvtColor(heatmap, cv2.COLOR_BGR2RGB)

    #      heatmap  alpha blending
    overlay = heatmap * alpha + img * (1 - alpha)
    overlay = overlay.astype(np.uint8)

    plt.figure(figsize=(8,8))
    plt.imshow(overlay)
    plt.title("CAM Overlay on Image")
    plt.axis('off')
    plt.show()
```

[27]:
```
#
orig_img = unnormalize(image)

# CAM
```

```
visualize_cam(cam)

#        CAM overlay
visualize_cam_on_image(cam, orig_img)
```





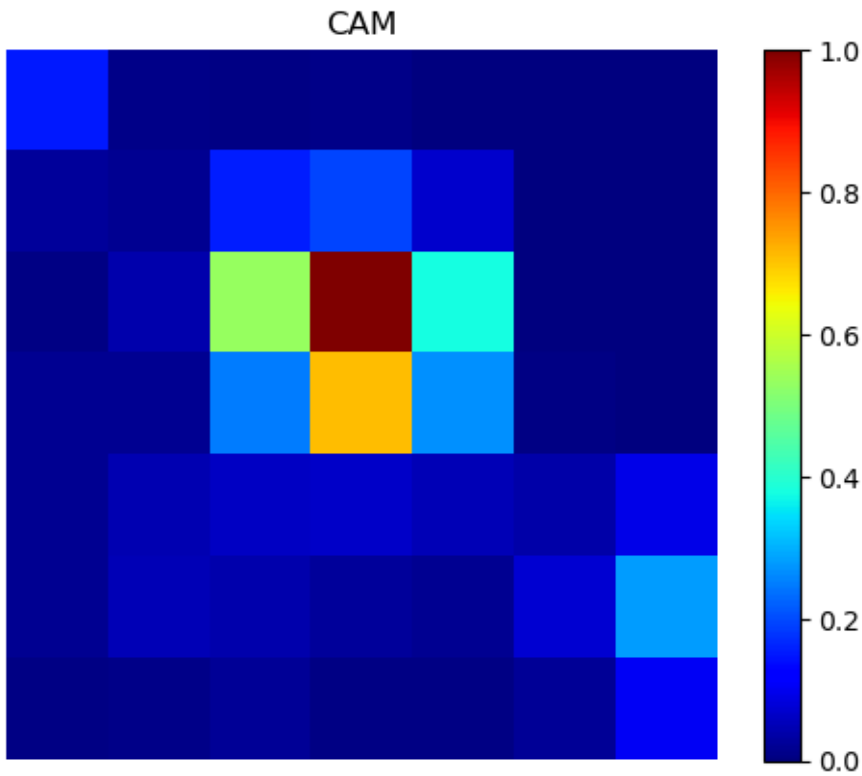
0.0.1 Grad-CAM

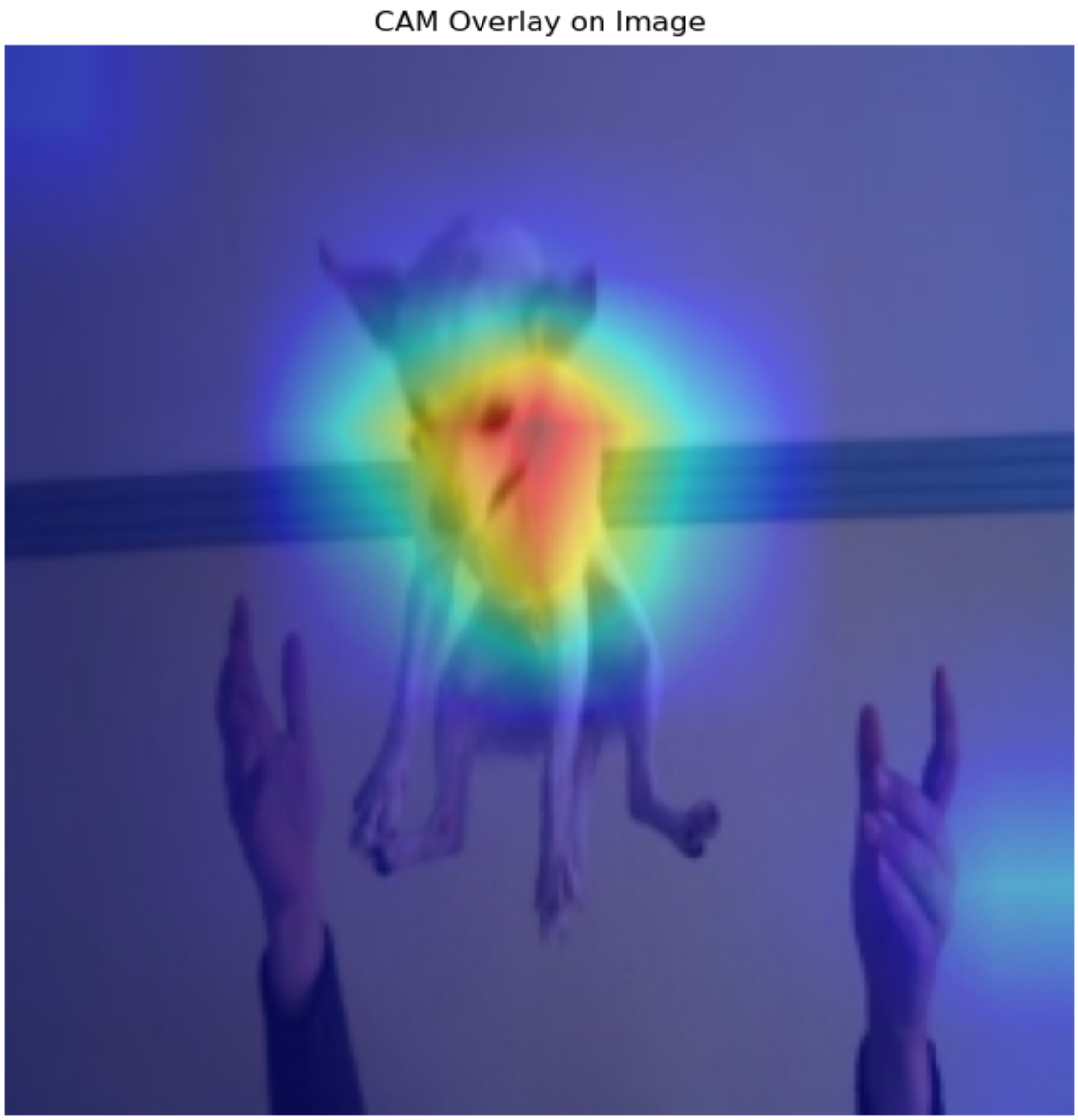
[28]:
```
def generate_grad_cam(model, image, target_layer_name):
    model.eval()
    features = {}
    gradients = {}

    # forward hook:
    def forward_hook(module, input, output):
        features['value'] = output.detach()

    # backward hook:      gradient
```

```
    def backward_hook(module, grad_in, grad_out):
        gradients['value'] = grad_out[0].detach()

    #         target_layer_name
    target_layer = dict(model.named_modules()).get(target_layer_name, None)
    if target_layer is None:
        raise ValueError(f"Layer '{target_layer_name}' not found in the model.")

    # hook
    forward_handle = target_layer.register_forward_hook(forward_hook)
    backward_handle = target_layer.register_backward_hook(backward_hook)

    #
    output = model(image)
    pred_class = output.argmax(dim=1).item()

    #         backward
    model.zero_grad()
    score = output[0, pred_class]
    score.backward()

    # hook
    forward_handle.remove()
    backward_handle.remove()

    #    feature map  gradient    (shape: [C, H, W])
    fmap = features['value'][0]
    grads = gradients['value'][0]

    #        gradient  global average pooling    (weight   )
    weights = torch.mean(grads, dim=(1, 2))

    # weighted sum:      feature map  weight
    grad_cam = torch.zeros(fmap.shape[1:], dtype=torch.float32, device=fmap.
↪device)
    for i, w in enumerate(weights):
        grad_cam += w * fmap[i, :, :]
    grad_cam = grad_cam.cpu().numpy()

    # ReLU       :         [0,1]
    grad_cam = np.maximum(grad_cam, 0)
    grad_cam = (grad_cam - grad_cam.min()) / (grad_cam.max() - grad_cam.min() +␣
↪1e-8)

    return grad_cam
```

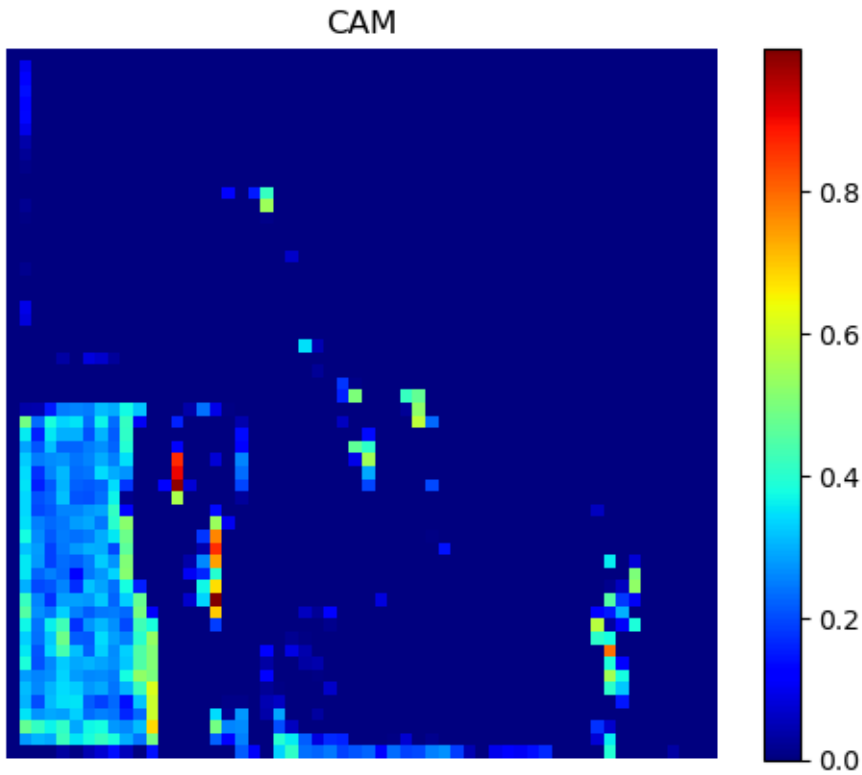
[29]:
```
# layer4
grad_cam = generate_grad_cam(model, sample_image, target_layer_name="layer4")

# Grad-CAM
visualize_cam(grad_cam)

#                 overlay
orig_img = unnormalize(image)
visualize_cam_on_image(grad_cam, orig_img)
```

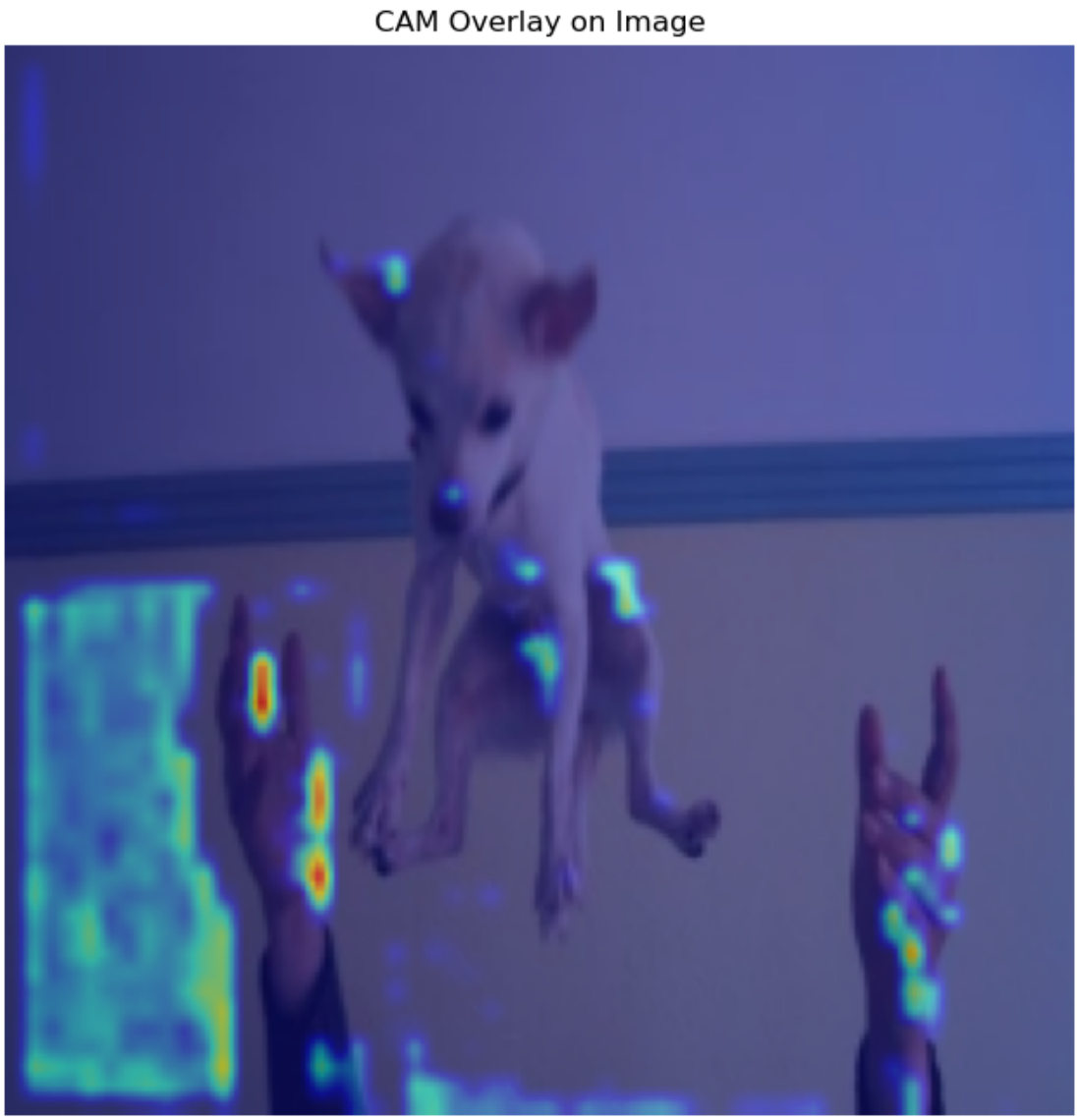
```
/opt/conda/lib/python3.12/site-packages/torch/nn/modules/module.py:1842:
FutureWarning: Using a non-full backward hook when the forward contains multiple
autograd Nodes is deprecated and will be removed in future versions. This hook
will be missing some grad_input. Please use register_full_backward_hook to get
the documented behavior.
  self._maybe_warn_non_full_backward_hook(args, result, grad_fn)
```



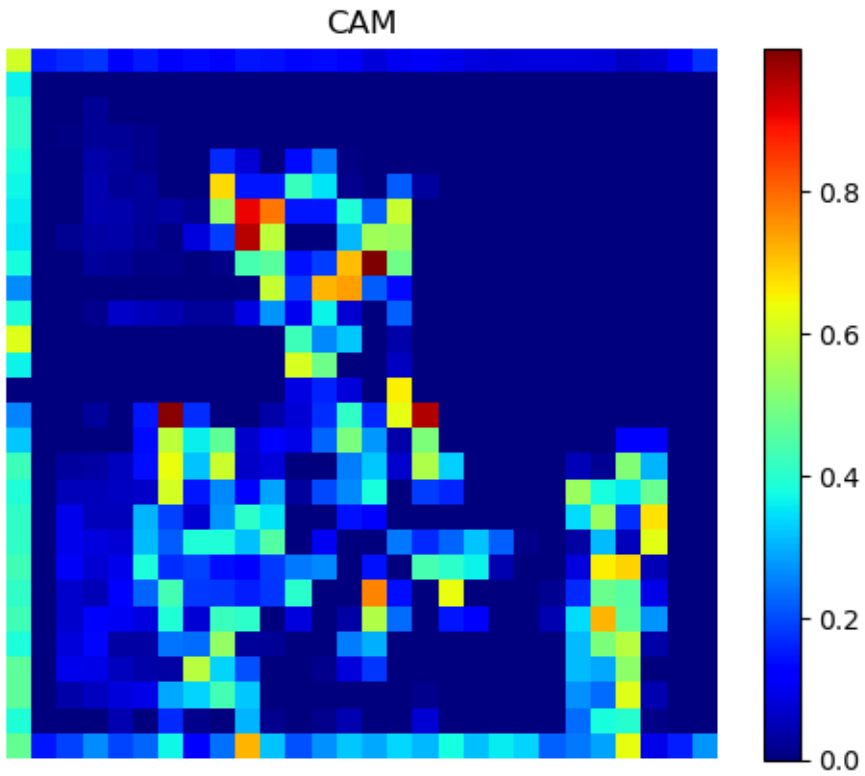


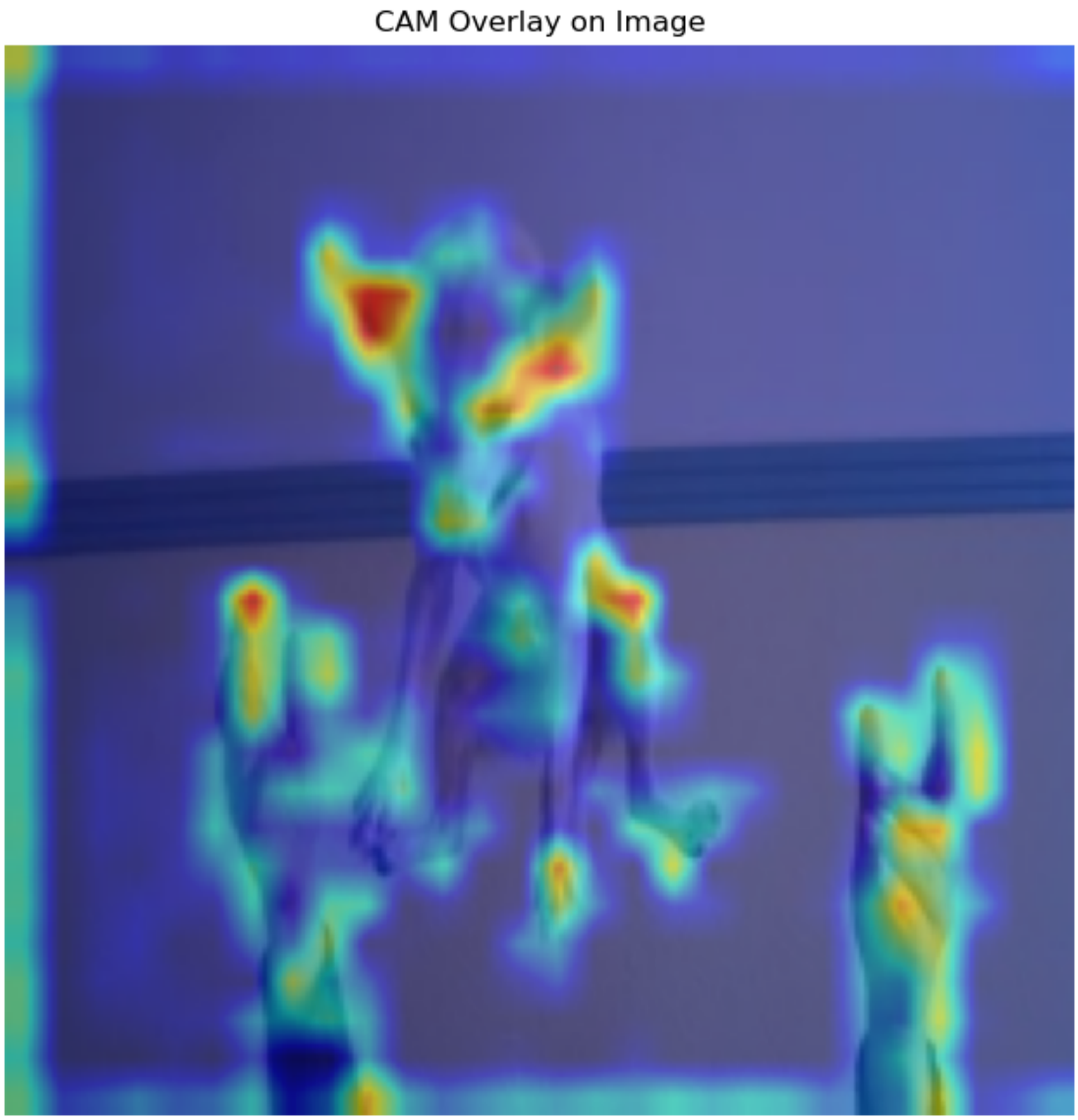
[30]:
```
grad_cam = generate_grad_cam(model, sample_image, target_layer_name="layer1")

# Grad-CAM
visualize_cam(grad_cam)

#                 overlay
orig_img = unnormalize(image)
visualize_cam_on_image(grad_cam, orig_img)
```


CAM
0.8
0.6
0.4
0.2
0.0

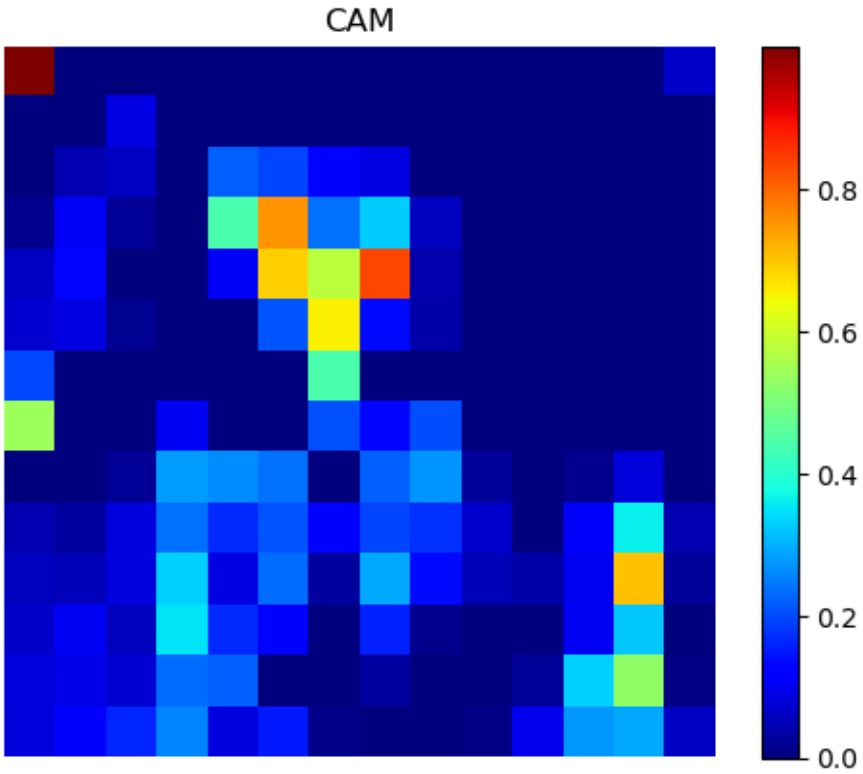


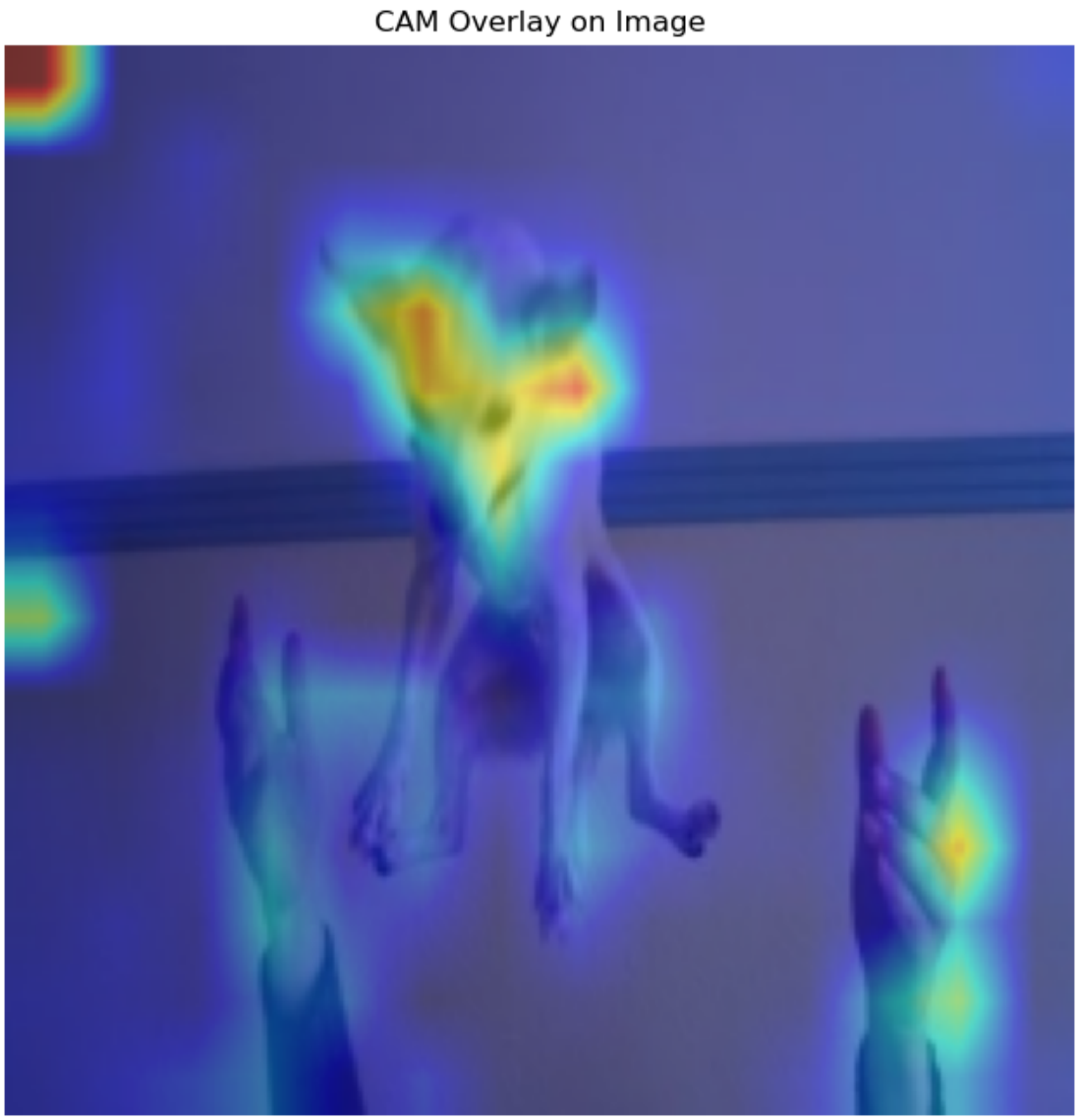
[31]:
```
grad_cam = generate_grad_cam(model, sample_image, target_layer_name="layer2")

# Grad-CAM
visualize_cam(grad_cam)

#                 overlay
orig_img = unnormalize(image)
visualize_cam_on_image(grad_cam, orig_img)
```


CAM
0.8
0.6
0.4
0.2
0.0



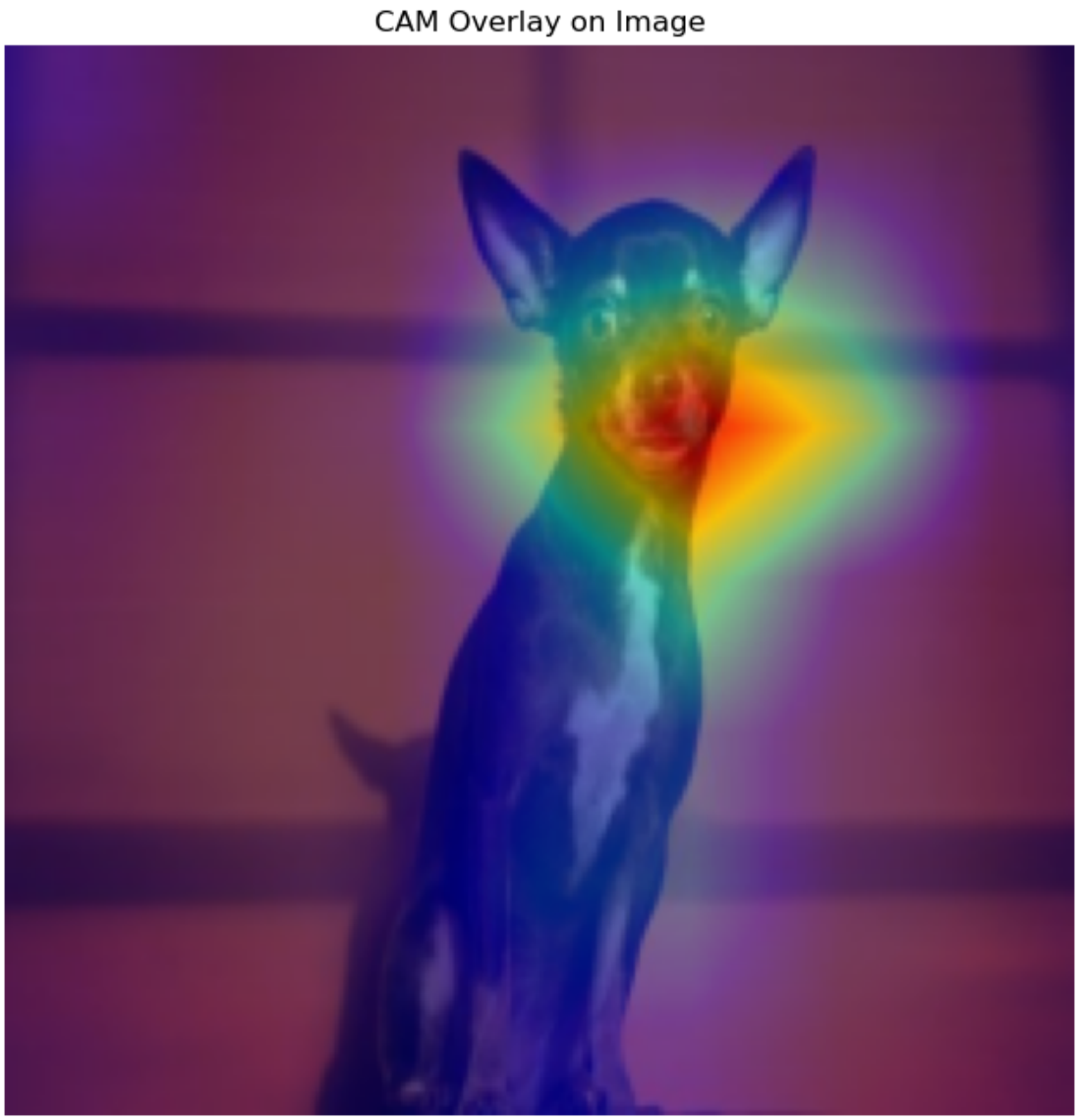
[32]:
```
grad_cam = generate_grad_cam(model, sample_image, target_layer_name="layer3")

# Grad-CAM
visualize_cam(grad_cam)

#                 overlay
orig_img = unnormalize(image)
visualize_cam_on_image(grad_cam, orig_img)
```


CAM
0.8
0.6
0.4
0.2
0.0



0.0.2 Detection with CAM

[33]:
```
image, label, ground_bbox = train_dataset[10]
sample_image = image.unsqueeze(0).to(device)
orig_img = unnormalize(image)
grad_cam = generate_grad_cam(model, sample_image, target_layer_name="layer4")

visualize_cam_on_image(grad_cam, orig_img)
```



[34]:
```
def get_bbox(cam, threshold=0.01):
    coords = np.argwhere(cam > threshold)
    if coords.size == 0:
        return None
    # np.argwhere    (row, col)  , (y, x)    .
    y_min, x_min = coords.min(axis=0)
    y_max, x_max = coords.max(axis=0)
    return (x_min, y_min, x_max, y_max)
```

[35]:
```
cam_resized = cv2.resize(cam, (orig_img.shape[1], orig_img.shape[0]))
bbox = get_bbox(cam_resized, threshold=0.5)
```

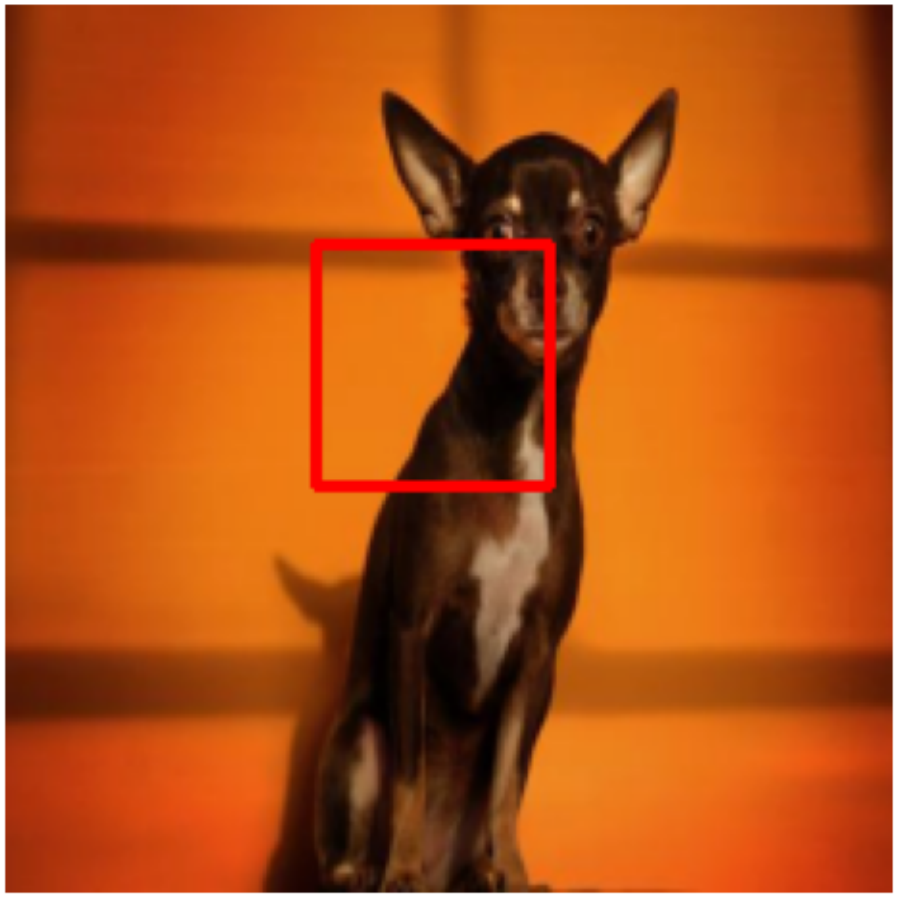
```
print(bbox)
```

```
(np.int64(78), np.int64(60), np.int64(137), np.int64(121))
```

[36]:
```
def visualize_bbox_on_image(img, bbox, box_color=(255, 0, 0), thickness=2):
    #
    img_with_bbox = img.copy()
    if bbox is not None:
        x_min, y_min, x_max, y_max = bbox
        cv2.rectangle(img_with_bbox, (x_min, y_min), (x_max, y_max), box_color,␣
 ↪thickness)
    else:
        print("        .")
    return img_with_bbox
```

[37]:
```
img_bbox = visualize_bbox_on_image(orig_img, bbox)

plt.figure(figsize=(6,6))
plt.imshow(img_bbox)
plt.title("Image with Bounding Box from CAM")
plt.axis("off")
plt.show()
```

Image with Bounding Box from CAM



[38]:
```
ground_bbox
```

[38]: [78.848, 19.333333333333332, 169.792, 222.0]

[39]:
```
def get_iou(bbox1, bbox2):
    # bbox      None  IoU 0.0
    if bbox1 is None or bbox2 is None:
        return 0.0

    x_min1, y_min1, x_max1, y_max1 = bbox1
    x_min2, y_min2, x_max2, y_max2 = bbox2

    #   bbox
    x_min_inter = max(x_min1, x_min2)
    y_min_inter = max(y_min1, y_min2)
    x_max_inter = min(x_max1, x_max2)
    y_max_inter = min(y_max1, y_max2)
```

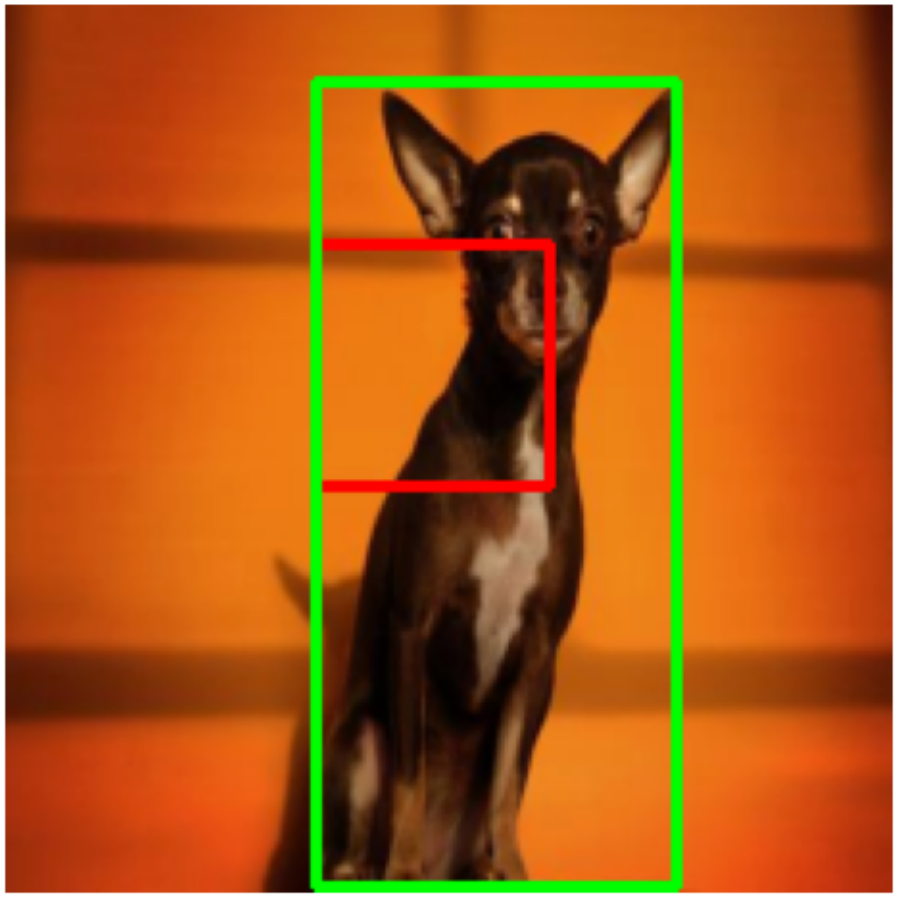
```
    #         (       )
    inter_width = max(0, x_max_inter - x_min_inter)
    inter_height = max(0, y_max_inter - y_min_inter)
    inter_area = inter_width * inter_height

    #   bbox
    area1 = (x_max1 - x_min1) * (y_max1 - y_min1)
    area2 = (x_max2 - x_min2) * (y_max2 - y_min2)

    #     : A   B = A + B - A   B
    union_area = area1 + area2 - inter_area
    if union_area <= 0:
        return 0.0

    iou = inter_area / union_area
    return iou
```

[40]:
```
get_iou(bbox, ground_bbox)
```

[40]: np.float64(0.191920321355413)

[41]:
```
def visualize_both_bbox_on_image(img, bbox, ground_truth, thickness=2):
    #
    img_with_bbox = img.copy()

    x_min, y_min, x_max, y_max = bbox
    cv2.rectangle(img_with_bbox, (x_min, y_min), (x_max, y_max), (255, 0, 0),␣
 ↪thickness)

    x_min_t, y_min_t, x_max_t, y_max_t = ground_truth
    cv2.rectangle(img_with_bbox, (x_min_t, y_min_t), (x_max_t, y_max_t), (0,␣
 ↪255, 0), thickness)

    return img_with_bbox

ground_truth = [int(x) for x in ground_bbox]

img_bbox = visualize_both_bbox_on_image(orig_img, bbox, ground_truth)

plt.figure(figsize=(6,6))
plt.imshow(img_bbox)
plt.title("Compare with Ground Truth")
plt.axis("off")
plt.show()
```

Compare with Ground Truth



0.0.3 : CAM

[42]:
```
import torch
import numpy as np
import cv2
import PIL

print(torch.__version__)
print(np.__version__)
print(cv2.__version__)
print(PIL.__version__)
```

```
2.7.1+cu118
2.2.6
4.13.0
11.2.1
```

[43]:
```
# CAM
def generate_cam(model, item):
    cam_image = None
    # TODO: generate cam image
    return cam_image
```

[44]:
```
cam_image = generate_cam(cam_model, item)
plt.imshow(cam_image)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[44], line 1
----> 1 cam_image = generate_cam(cam_model, item)
      2 plt.imshow(cam_image)

NameError: name 'cam_model' is not defined
```

[46]:
```
def visualize_cam_on_image(cam, img, alpha=0.5):

    # TODO: blend image

    plt.figure(figsize=(8,8))
    plt.imshow(overlay)
    plt.title("CAM Overlay on Image")
    plt.axis('off')
    plt.show()
```

[47]:
```
def generate_grad_cam(model, activation_layer, item):
    grad_cam_image = None
    # TODO: generate grad_cam_image

    return grad_cam_image
```

[48]:
```
def get_iou(gt_bbox, pred_bbox):
    iou = None
    # TODO: get iou between two bbox
    return iou
```

[ ]:
```

```